

Entregable semana 5

1. ¿Por qué el RDS no tiene IP pública, y cómo llega entonces la EC2 a él?

RDS se lanza sin IP pública porque una base de datos no tiene por qué estar accesible desde internet: exponerla directamente haría que atacantes puedan tumbar la conexión.

La EC2 llega a él porque ambos viven en la misma VPC . La conexión se hace por la IP/ endpoint privado del RDS, y lo que realmente controla el acceso es el security group de la base de datos: se configura para aceptar tráfico en el puerto de la base de datos únicamente cuando el origen es el security group de la EC2, no una IP ni un rango abierto. Así, solo la instancia que corre la API puede conectarse, y nadie desde fuera de la VPC puede siquiera intentarlo.

2. ¿Qué habría pasado si dejabas el SSH abierto al mundo?

Con el puerto 22 abierto a 0.0.0.0/0 , cualquier IP en internet puede intentar conectarse por SSH a la instancia. En la práctica esto significa que, en minutos u horas, bots automatizados empiezan a hacer port scanning y ataques de fuerza bruta contra el puerto 22, probando miles de combinaciones de usuario/contraseña o intentando explotar vulnerabilidades conocidas del servicio SSH. Si logran entrar (por una clave filtrada, una contraseña débil, o una vulnerabilidad sin parchar), podrían usar la instancia para minar criptomonedas, lanzar ataques a terceros, robar datos o credenciales almacenadas (como las de ~/.aws/credentials), o pivotar hacia otros recursos de la cuenta si la instancia tiene un rol IAM con permisos amplios. Por eso la regla del día era limitar el origen del puerto 22 a “My IP”: reduce la superficie de ataque a un único origen conocido.

3. ¿Qué midió tu comparación de query contra scan?

Medí el costo real de cada operación usando --return-consumed-capacity TOTAL , comparando tres números: Count (elementos devueltos), ScannedCount (elementos que DynamoDB tuvo que leer para responder) y CapacityUnits (lo que efectivamente se cobra).

El resultado mostró que query por taskId leyó exactamente los 3 elementos de esa partición y solo esos (Count = ScannedCount = 3 , costo 0.5 CU), porque

la partition key le permite a DynamoDB ir directo a los datos relevantes. En cambio, scan con un filtro por tipo = COMPLETADA tuvo que leer los 5 elementos de la tabla completa (ScannedCount = 5) para devolver solo 1 (Count = 1), con un costo mayor (2.0 CU) a pesar de traer menos resultados. La lección es que el filtro de un scan no reduce lo que se lee ni lo que se cobra: DynamoDB lee la tabla entera y descarta después lo que no cumple el filtro. Con cinco elementos la diferencia es pequeña, pero con millones de elementos un query sigue costando lo mismo mientras un scan se vuelve proporcionalmente carísimo.

Resultados medidos:

	Count	ScannedCount	CapacityUnits
query por taskId (T-001)	3	3	0.5
scan con filtro (tipo = COMPLETADA)	1	5	2.0

4. ¿Qué hace cada pieza del pipeline y por qué elappspec.yml viaja dentro del artefacto?

CodePipeline es el orquestador: detecta el cambio en la rama main del repo (vía la conexión con GitHub) y encadena, en orden, las etapas Source -> Build -> Deploy, sin que nadie tenga que ejecutar nada manualmente.

CodeBuild, guiado por buildspec.yml , es quien compila: instala el runtime (Corretto 21), corre mvn package , renombra el jar a un nombre estable (taskflow-api.jar) y empaqueta en un zip el jar junto con appspec.yml , taskflow.service y la carpeta scripts/ .

El bucket de S3 es el almacén intermedio: ahí es donde CodeBuild deja el zip (el artefacto) y de ahí es de donde CodeDeploy y el agente dentro de la EC2 lo descargan. Es el punto de entrega entre “construir” y “desplegar”.

CodeDeploy, guiado por appspec.yml , es quien despliega dentro de la instancia: le dice al agente instalado en la EC2 dónde copiar el jar y el archivo de unidad de systemd, y en qué momento ejecutar cada script (ApplicationStop , AfterInstall , ApplicationStart , ValidateService).

El appspec.yml tiene que viajar dentro del artefacto (el zip) porque CodeDeploy no lee el repositorio de GitHub directamente: solo conoce el contenido del zip

que CodeBuild dejó en S3. Si appspec.yml no estuviera listado en artifacts.files de buildspec.yml , el agente no tendría ninguna instrucción de qué hacer con el jar una vez descargado, aunque el archivo sí existiera en el repo.

5. ¿En qué hook vive cada comando que el miércoles corriste a mano?

Comando de ayer (a mano)	Script de hoy	Hook de CodeDeploy
kill <PID>	scripts/parar.sh	ApplicationStop
chown / ajustar permisos	scripts/permisos.sh	AfterInstall
nohup java -jar ... &	scripts/arrancar.sh	ApplicationStart
Abrir el navegador a comprobar que respondía	scripts/verificar.sh	ValidateService

En el primer despliegue, ApplicationStop se salta porque no hay una revisión anterior de la cual “parar” nada; a partir del segundo despliegue sí corre, y por eso el script incluye `|| true` para que, si el servicio aún no existe, `systemctl stop` no tumbe todo el despliegue con `set -e` activo.